

Assignment 2: On-Demand Home Services Marketplace Platform

Assignment Objective:

The objective of this assignment is to familiarize ourselves with the concepts provided by EJBs, RabbitMQ, and microservices by building an On-Demand Home Services Marketplace Platform.

System Description:

Build a distributed on-demand home services marketplace where customers can book skilled professionals (plumbers, carpenters, electricians, cleaners, painters, etc.) for service requests. Service providers offer services with specific prices and available dates. Customers can search for available service by category and book a suitable service. Every customer has a wallet. When a customer makes a booking, there must be sufficient balance in their wallets.

The system demonstrates microservices, EJBs, and asynchronous message processing using RabbitMQ with REST API integration for all operations.

Functional Requirements:

Admins in the system should have the following abilities:

1. View all registered users
2. View all transaction history for all users (booking records)
3. Add a new service category

As a **service professional / service provider** (Plumber, Carpenter, etc.), you should be able to:

4. Register with profession type (e.g., 'Plumber', 'Carpenter') in addition to username and password
5. Login to the system using registered credentials
6. Create service offer with price (e.g., \$80 per plumbing repair) and available date
7. View all active service offers and their details.
8. Update a service offer pricing and availability
9. View completed services with their info including customer names
10. Receive and view booking information

As a **customer**, you should be able to:

1. Register as a new customer (username, password, balance)
2. Set initial wallet balance during registration (e.g., \$200)
3. Add funds to wallet at any time (e.g., add \$100 to existing \$200 balance)
4. View current wallet balance
5. Browse available services by category (plumbing, carpentry, electrical, etc.)
6. Book a professional service
7. Receive booking confirmation and status updates
1. View booking history (current and past bookings)

Example Scenario:

- A customer registers and adds \$200 to their wallet
- Customer search for a plumber
- System displays all available plumber services (previously offered by plumbers (service professional))
- The Customer chooses an offer – plumber offer with suitable pricing and available date.
- System verifies that the customer has sufficient balance
- Payment deducted from wallet (\$200 - \$80 = \$120 remaining)
- Both parties notified asynchronously via RabbitMQ

Use RabbitMQ to apply the following requirements:

- When a booking is made, system verifies that the Customer wallet has sufficient balance
- If available and balance confirmed: Deduct amount from wallet → Complete booking with all information → Send confirmation to the customer and the service provider
- If unavailable or insufficient balance: REJECT booking → ROLLBACK all actions → Return deducted amount to wallet. Send a notification to the customer
- All confirmations sent asynchronously
- All notifications are delivered through REST API endpoints that return notification data in JSON format.

Additional features for students working in teams of 3

- Customers can issue service requests with specific requirements (service category, price, required date)
- System shall match this service request with an available service offer with matching date and price.
- Using RabbitMQ, service providers should be notified of new customer requests matching their services.
- If a service provider accepted a customer request, a booking should be made with the same logic as before. And both the customer and provider shall be notified.

- Admins should be notified in case of payment failure. In case of only 'PaymentFailed' events on payments exchange (Hint: Requires use of RabbitMQ direct exchange feature)

Technical Requirements for the assignment (for all team sizes)

1. Using Microservices:

- The above system should be designed to follow the microservice architectural style.
- Each service should be implemented as its own project. This means that it has its own codebase and its own DB. If you have S1 Service and S2 Service then S1 shouldn't be able to get any information from the DB of S2, but instead should request it from the S2 Service through REST calls.

2. Using EJBs

- One of the services must be developed using EJBs while applying two EJB types.
- You are required to use any two of these 4 different bean types to fulfill some of the above functional requirements(Stateless, Stateful, Singleton, Message Driven)
- The remaining services can be developed in Java only, OR you can choose to develop only one of the services in a different programming language (or using a different framework like Spring)

Deliverables:

- The source code for your developed application
- Postman collection to test all the endpoints of all the function requirements.

Rules of Submission:

1. No email submissions will be accepted
2. There are no late submissions
3. The assignment is in groups of 2 ~3 with the same TA. In case you're forming a group of 3, then you MUST implement the additional features
4. If more than 3 team members submit the assignment, all team members will get zero
5. Cheating is not tolerated and will be given negative grades

6. You should submit your assignment as ONE zip file with the below naming convention: DS_Assign2_GroupNumber_ID1_ID2 (example: DS_Assign2_S1_20116001_20116002)
7. You should submit your source code along with a document explaining any decisions or assumptions that you made. This document should also include any steps needed so that the TA can properly run your code. It is your duty to write down the exact steps needed to run the source code properly
8. You **SHOULD NOT** copy any code from the internet or from your colleagues. It will be detected and considered as a cheating case
9. Submission of the assignment will be on Google Classroom through a Google form link that will be shared later
10. Deadline for the submission is **May 8th, 2026**. A separate submission link will be shared through the Google classroom